

雙核心 CPU 模擬系統

VGA 影像輸出與匯流排仲裁

Hneemann Digital

Custom ISA

VGA 640×480

Verilog HDL

2

獨立核心

6

自訂指令

256×3

bit VRAM



為什麼做這個專題？

01

排程與控制

現代多核 CPU 最難的不是運算，而是「誰先用資源」。想做一個簡化的排程器（Round-Robin Arbiter）來理解這個概念。

02

自訂 ISA

自己設計指令集，理解硬體從 opcode 解碼到 ALU 執行到 WRITEBACK 步驟。

03

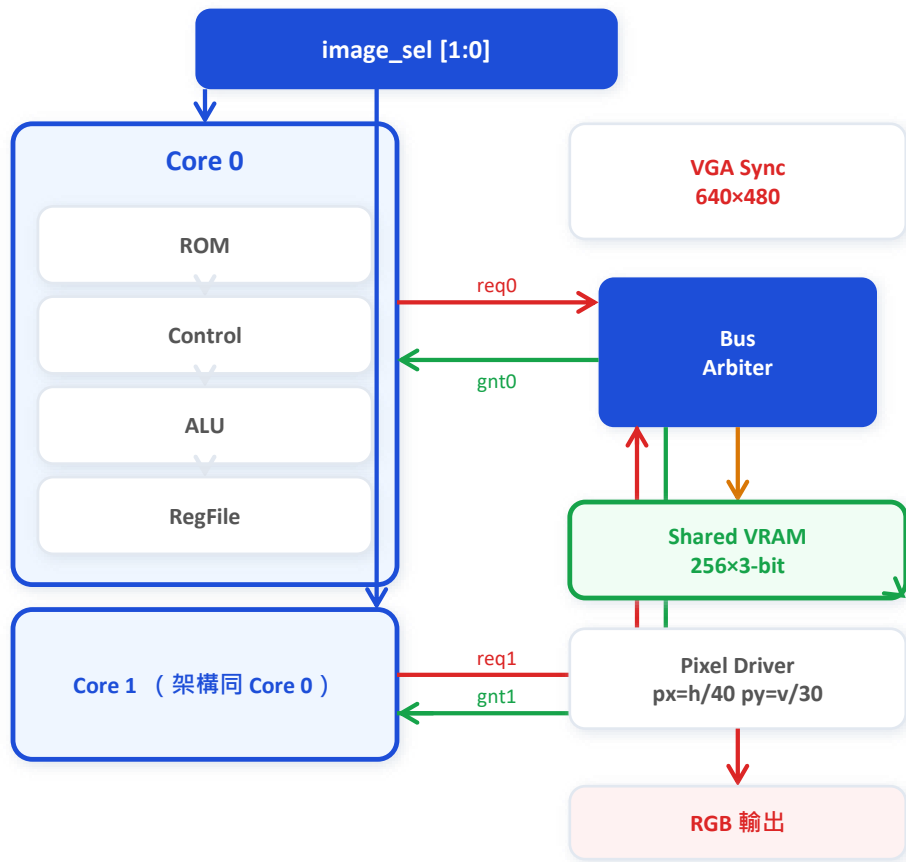
平行計算

兩個核心同時寫 VRAM、共用同一條匯流排——這正是多核系統最核心的問題：並行存取衝突與解決方法。



系統整體架構

六大模組，訊號由上而下流動



訊號流說明

①

image_sel 決定圖案

2-bit 輸入，同時送入兩個核心，決定跑哪一套程式（哪張圖）。

②

兩核各自執行指令

Core 0 負責 row 0-7，Core 1 負責 row 8-15，各自有獨立的 ROM、Control、ALU、RegFile。

③

OUT 指令請求匯流排

寫 VRAM 前先送 req 給 Bus Arbiter，等到拿到 gnt 才能寫入，防止同時撞寫。

④

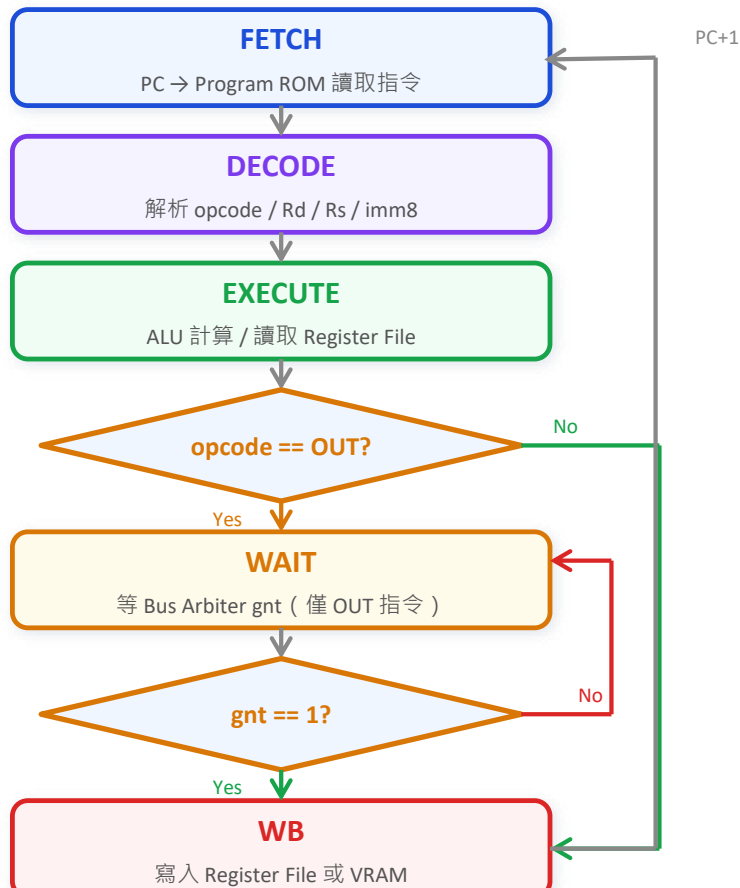
VGA 即時掃描

VGA Sync 持續產生 hcount/vcount，Pixel Driver 換算成 VRAM 位址讀出 RGB，送到螢幕。



Control Unit FSM

每個核心的控制邏輯



指令集 ISA (16-bit)

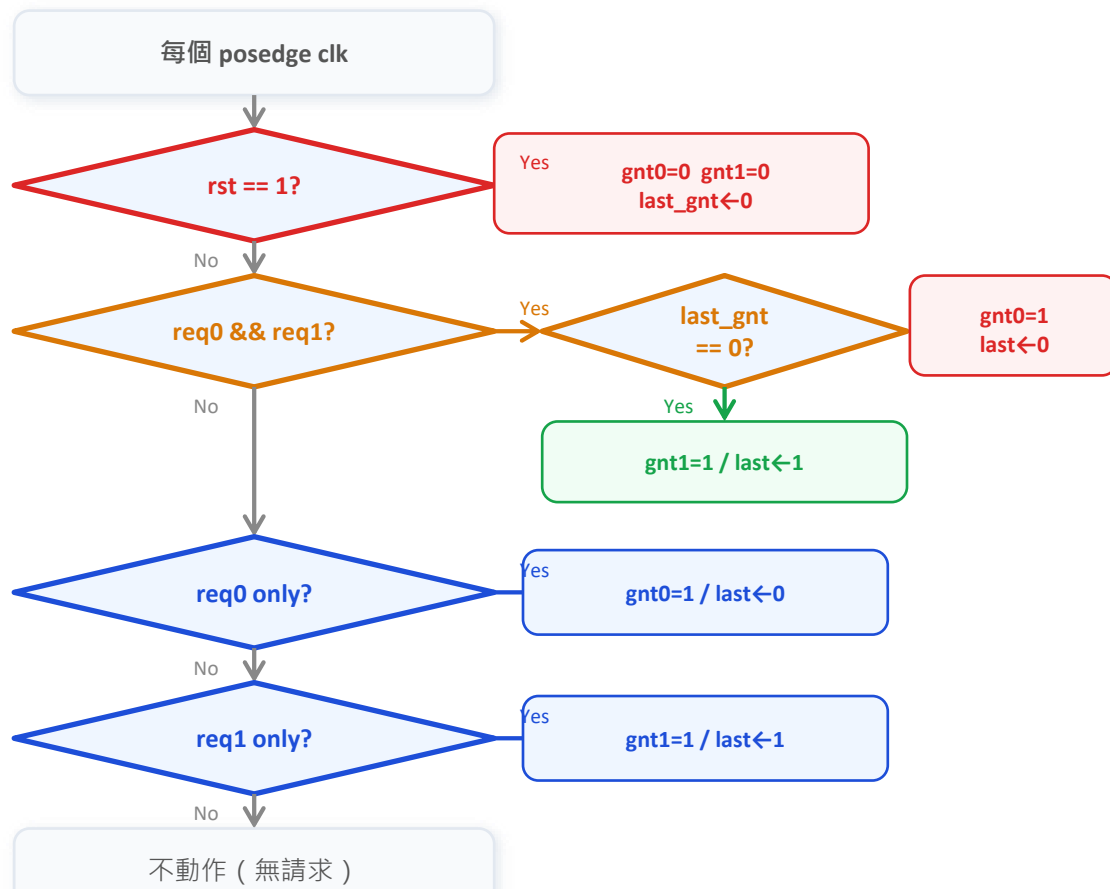
Opcode	指令	功能
0000	MOV	$Rd \leftarrow imm8$
0001	ADD	$Rd \leftarrow Rd + Rs$
0010	SUB	$Rd \leftarrow Rs - imm8$
0011	AND	$Rd \leftarrow Rs \& imm8$
0100	JMP	$PC \leftarrow imm8$
0101	OUT	$VRAM[imm8] \leftarrow Rd$

OUT 指令特殊路徑

執行後不直接 WRITEBACK，先進入 WAIT_GNT 等待 Bus Arbiter 授予 gnt，確保兩核不會在同一個 clock 寫入相同 VRAM 位址。



Bus Arbiter — Round-Robin 仲裁



設計重點

公平

Round-Robin 輪流

兩核交替取得授權，不會有任一核被餓死 (starvation) 的問題。

記憶

last_gnt 狀態

記錄上一次授予誰，決定這次同時請求時優先給誰。

防護

gnt 直接控 wr_en

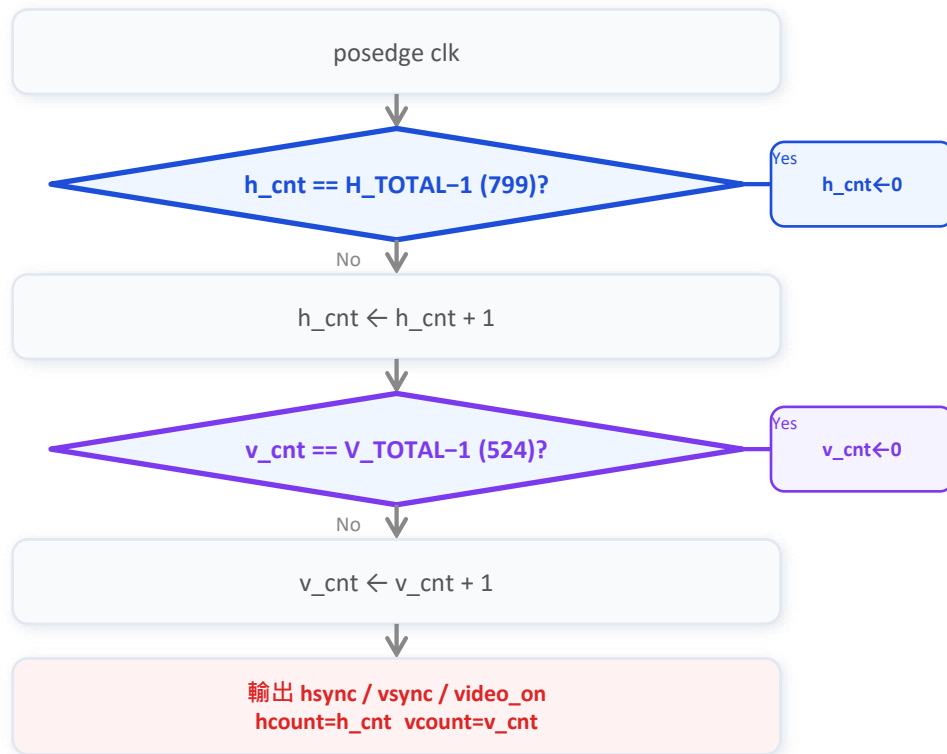


gnt0→VRAM wr_en0 ; gnt1→wr_en1
· 兩個 enable 物理上互斥。

VGA 輸出 × Pixel Driver

如何從 VRAM 讀出顏色打到螢幕

VGA Sync 計數器



Pixel Driver 座標映射



每個 VRAM 格 = 螢幕上 40×30 像素的色塊 (16×16 格鋪滿 640×480)

Demo 時間

切換到 Hneemann Digital

① 電路功能驗證

- image_sel 切換 → 兩個核心同步更換圖案
- Reset 後兩核從 PC=0 重新開始
- VGA 輸出 16×16 色塊 (上半 Core 0 / 下半 Core 1)

② 波形圖觀察

- req0 / req1 — 兩核何時送出寫入請求
- gnt0 / gnt1 — 永遠不同時為 1
- 同時 req 時 gnt 交替 → Round-Robin 正確運作